Okänd upphovsman, *Intersection deadlock*. Ryssland. 2012.

Synkronisering:
Deadlocks

Jimmy Åhlander

2023-11-17

Mittuniversitetet
MID SWEDEN UNIVERSITY

Thue, *Car crash*. Köpenhamn, Danmark. 2005.

Agenda

- Deadlocks:
  - Definition.
  - De fyra kraven.
  - Hur man förhindrar och hanterar dem.
- Dining philosophers problem.
- Exempel i Python.

Deadlock (def.)

- En deadlock kan uppstå i multiprogrammingmiljöer där flera processer konkurrerar om någon eller några resurser.

- Om processerna som för tillfället håller resurserna aldrig släpper dem kan ett oändligt väntande uppstå: en deadlock.

- Uppfyller vi fyra givna krav så *kan* en deadlock uppstå.

Krav för att en deadlock ska uppstå

1. **Mutual exclusion**
   Det måste finnas åtminstone en resurs som kan låsas för användning av endast en process (det går alltså inte att dela resursen).

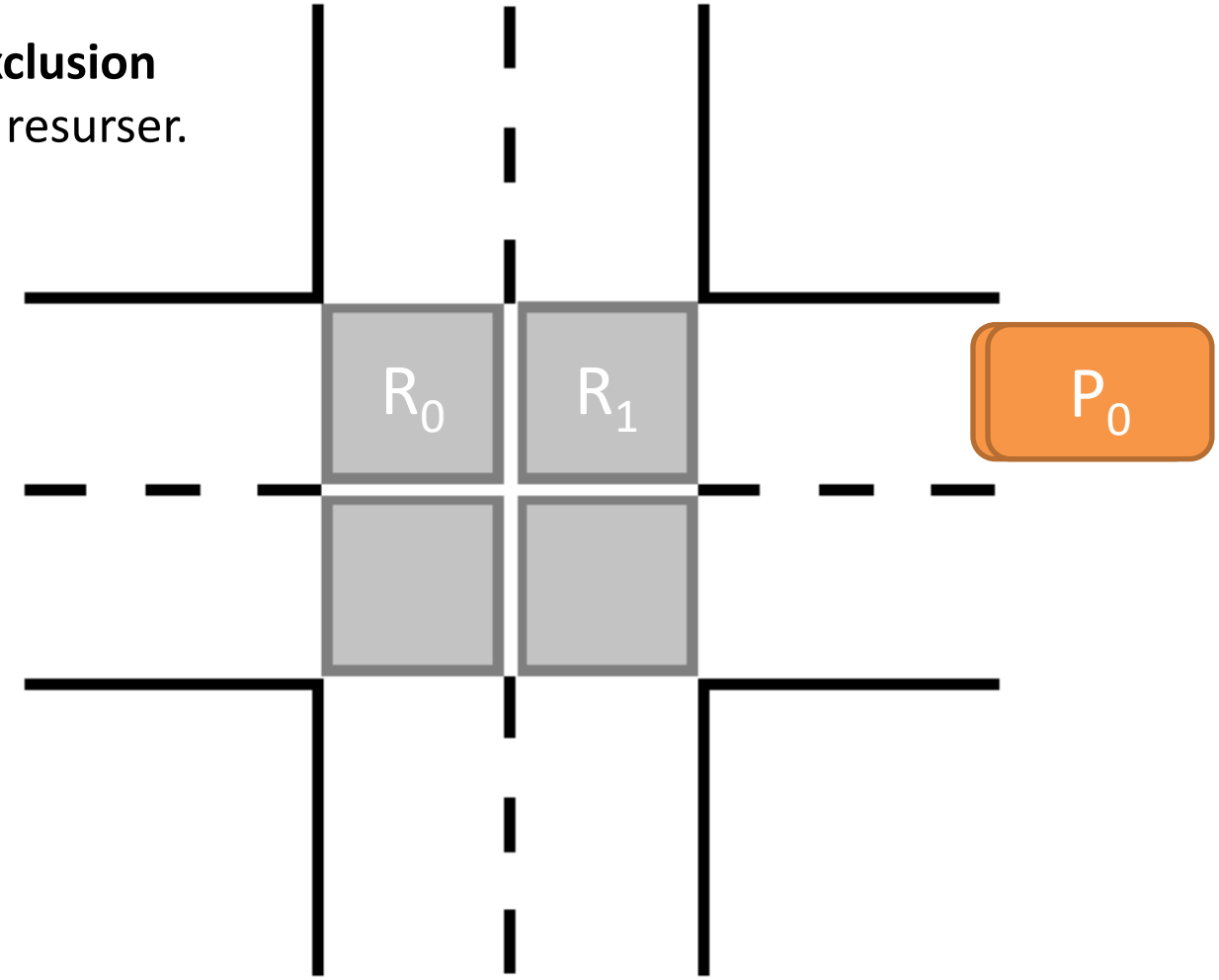
2. **Hold and wait**
   Alla processer i deadlocken måste hålla en resurs och samtidigt vänta på en annan resurs som hålls av en annan process.

3. **No preemption**
   En process kan inte tvingas släppa sin resurs.

4. **Circular wait**
   Alla processer i deadlocken måste vänta på varandra på ett sådant sätt att de formar en cirkel. D.v.s. med N antal processer väntar P_0 på P_1, P_1 på P_2, …, och till sist P_{N-1} på P_0.

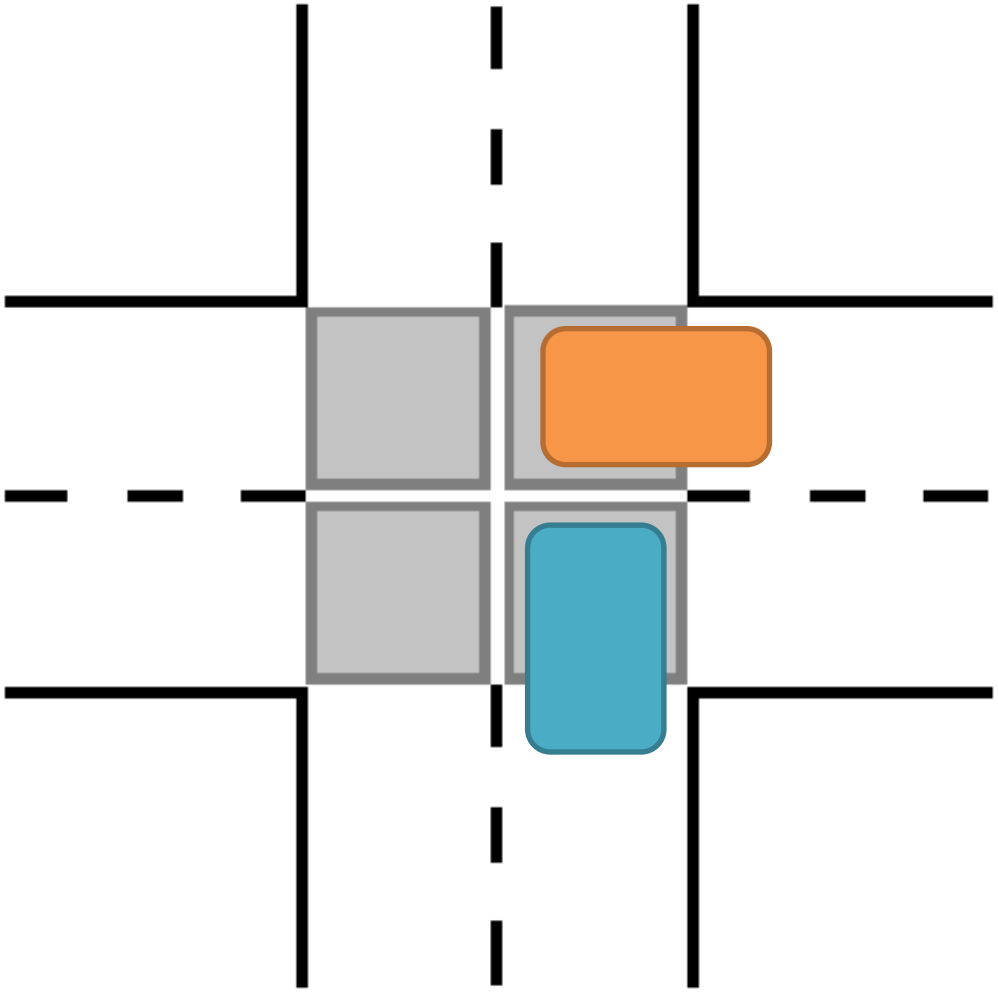
Kraven visualiserade

1. **Mutual exclusion**
   Ej delbara resurser.



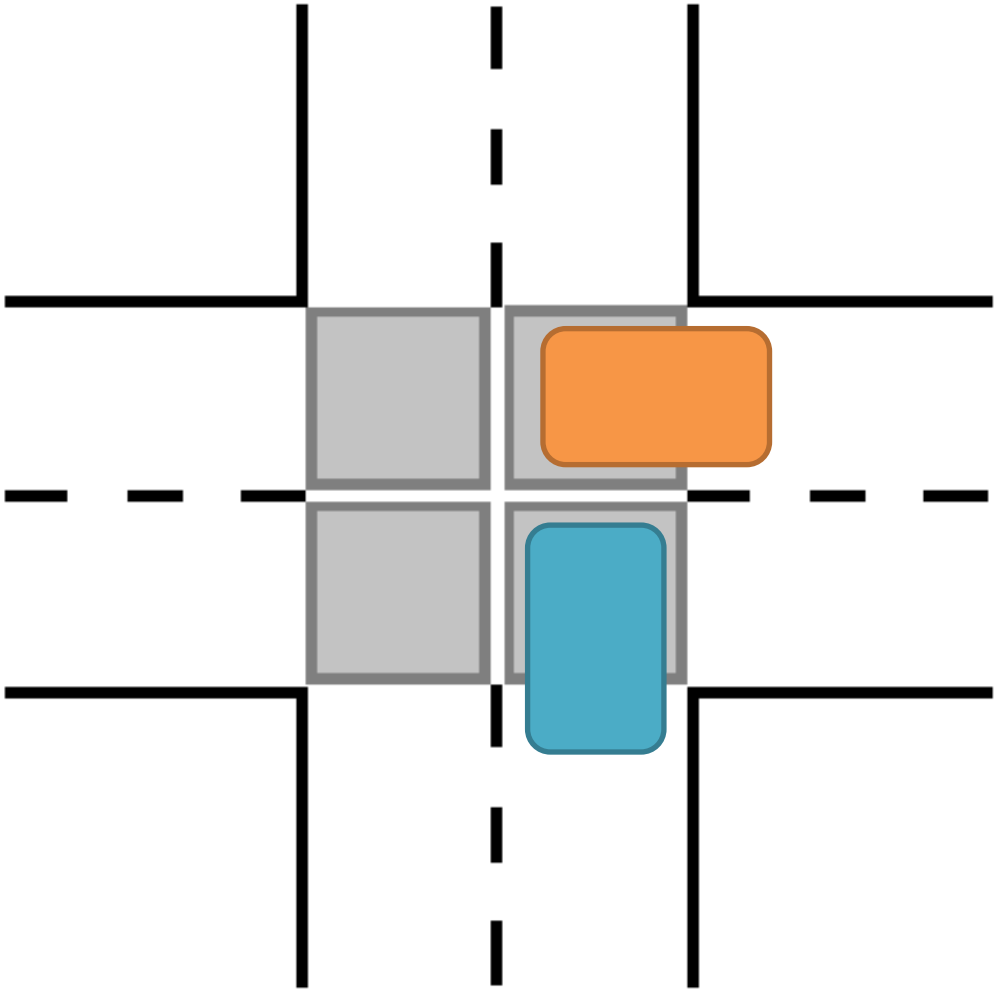
Kraven visualiserade

2. **Hold and wait**
   Håll på en resurs och vänta på en annan.



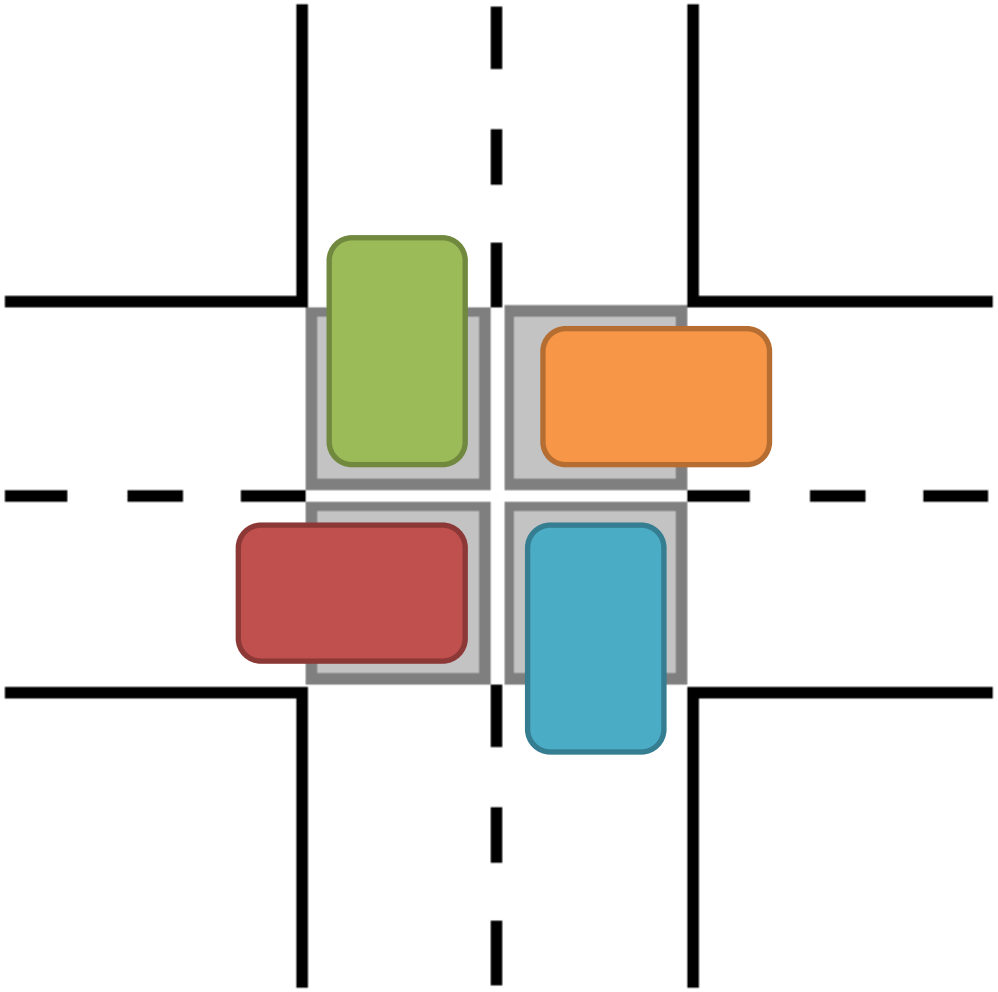
Kraven visualiserade

3. **No preemption**
   Kan inte bli tvingad att släppa sin resurs.



Kraven visualiserade

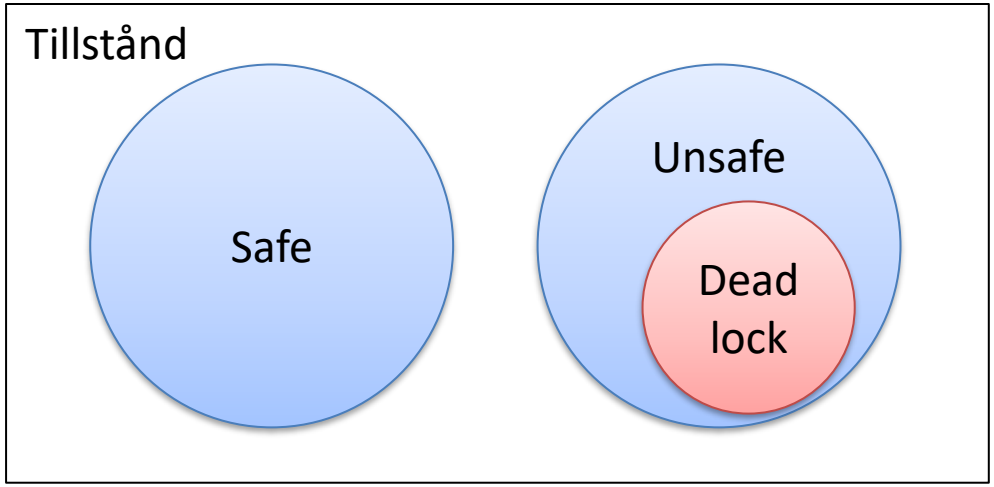
4. **Circular wait**
   Alla väntar i en cirkel.



Förhindra en deadlock?

- Alla fyra krav måste uppfyllas för att en deadlock ska kunna uppstå. Förhindras ett valfritt krav så kan en deadlock inte uppstå.

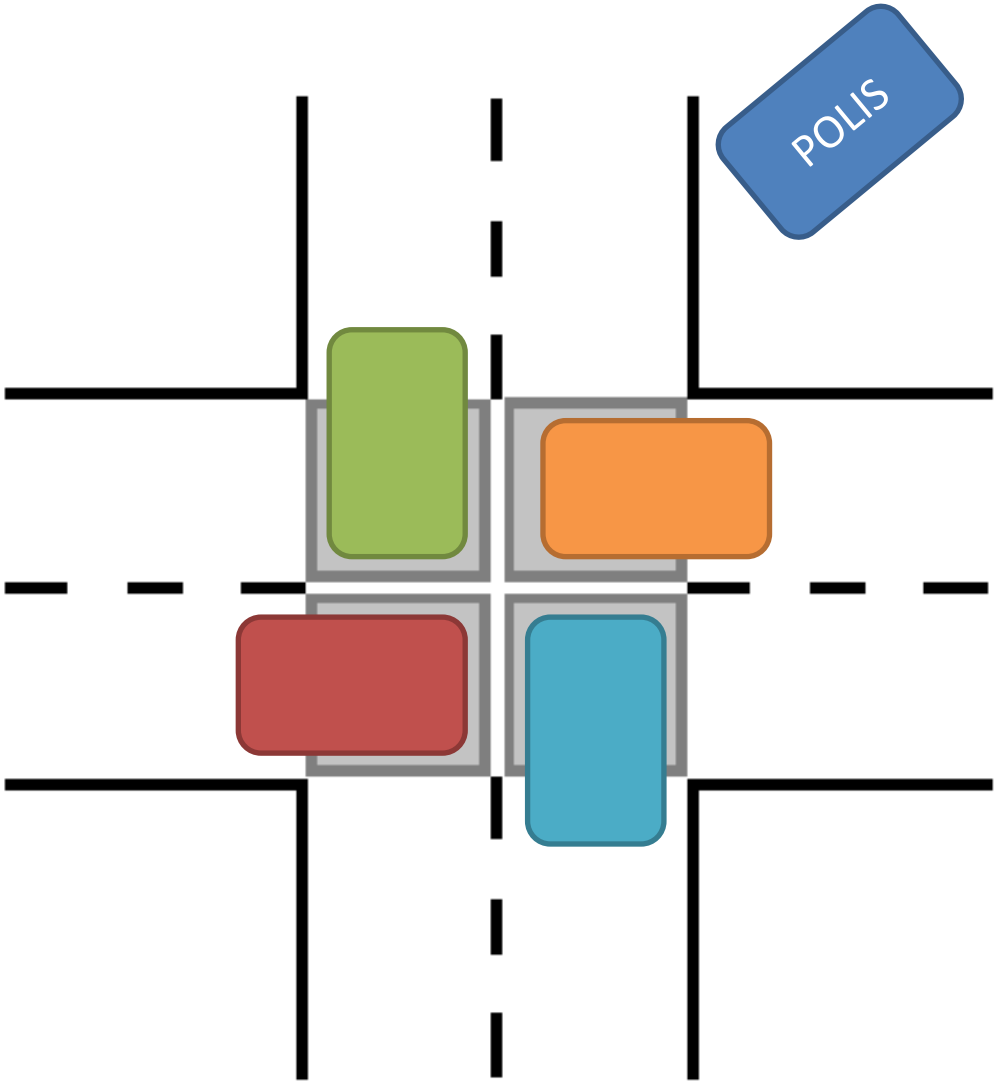
- Även om alla fyra krav kan uppfyllas så är det inte en garanti för att en deadlock kommer att uppstå.



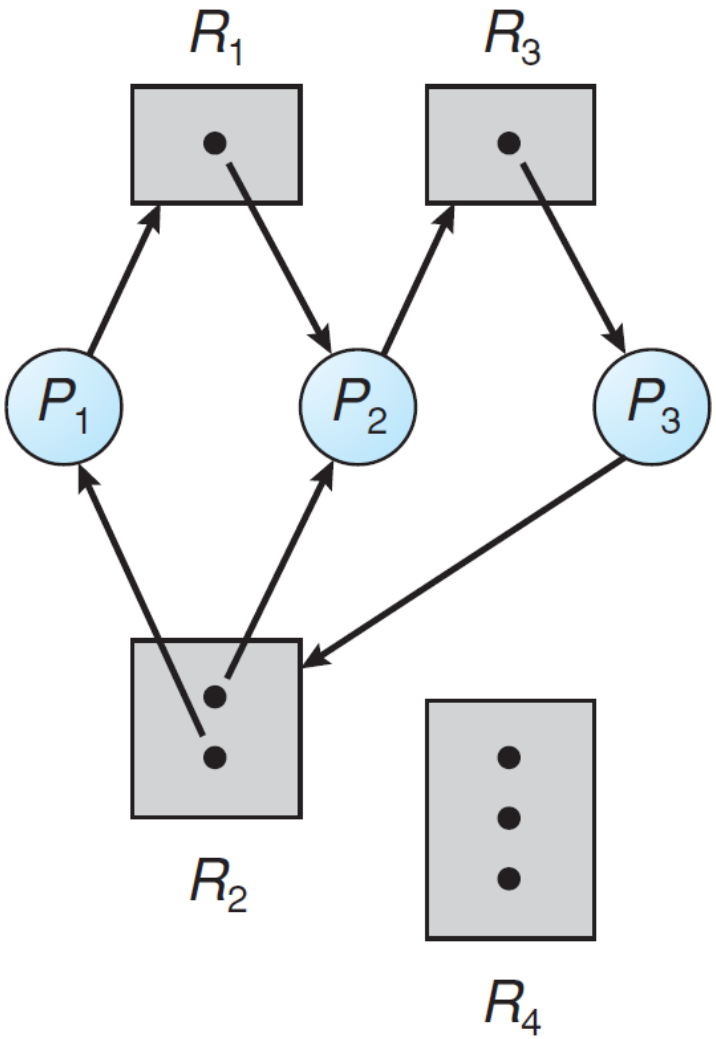
Förhindra en deadlock (visualiserat)

Stoppa:

1. **Mutual exclusion**
   Delbara resurser.
2. **Hold and wait**
   Håll inte resurser samtidigt som andra inväntas.
3. **No preemption**
   Låt någon stoppa.
4. **Circular wait**
   Få bort cirkeln.

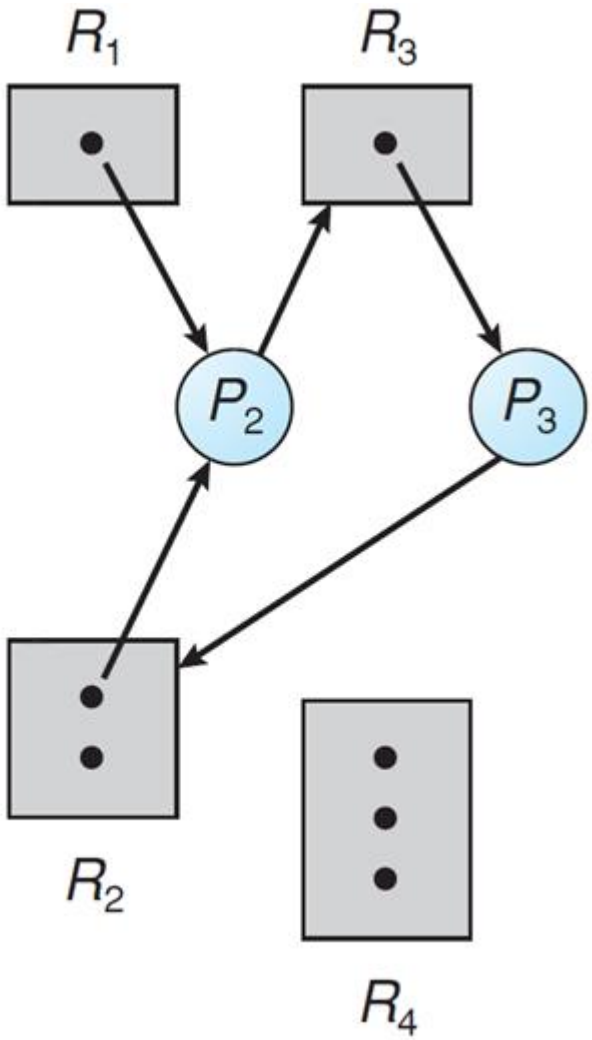


Deadlock (ex.)



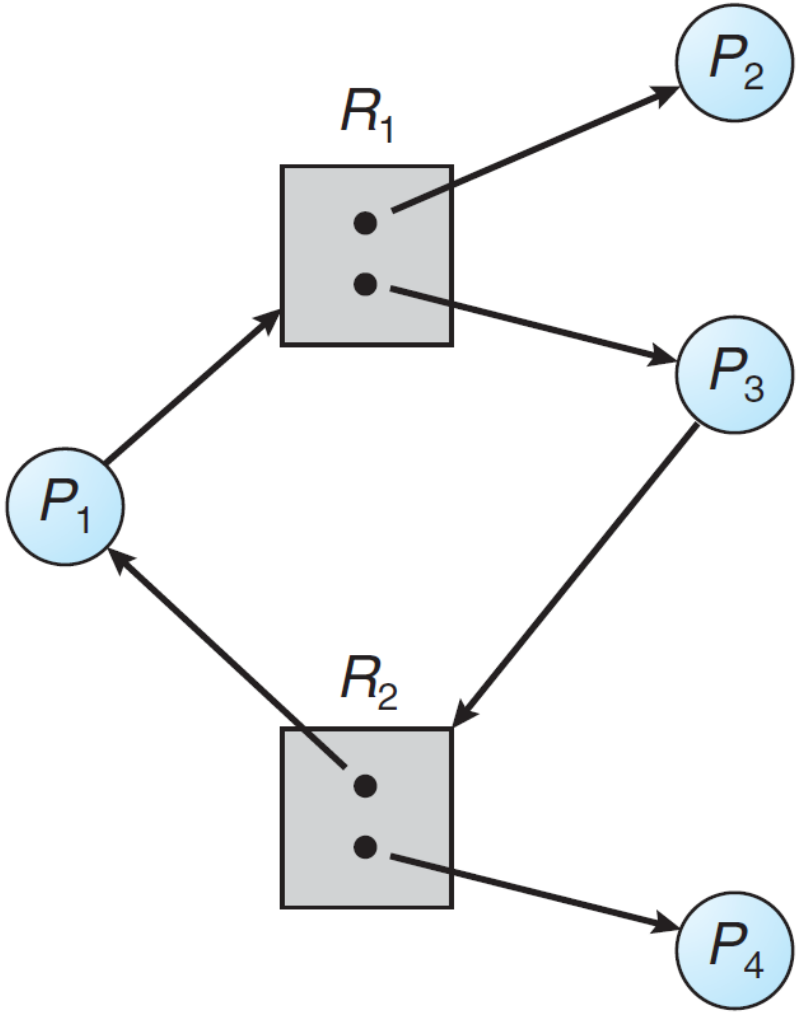
A. Silberschatz, et al. Operating System Concepts. International Student Version. 9th ed. Hoboken, N.J.: John Wiley & Sons Inc, 2013. s. 317

Ej deadlock (ex.)



Se föregående slide. *Modifierad*.

Ej deadlock (ex.)



A. Silberschatz, et al. Operating System Concepts. International Student Version. 9th ed. Hoboken, N.J.: John Wiley & Sons Inc, 2013. s. 317

Tre olika sätt att hantera deadlocks

1. Använda ett protokoll eller en algoritm som förhindrar eller undviker deadlocks helt i systemet, t.ex. Banker's algorithm.

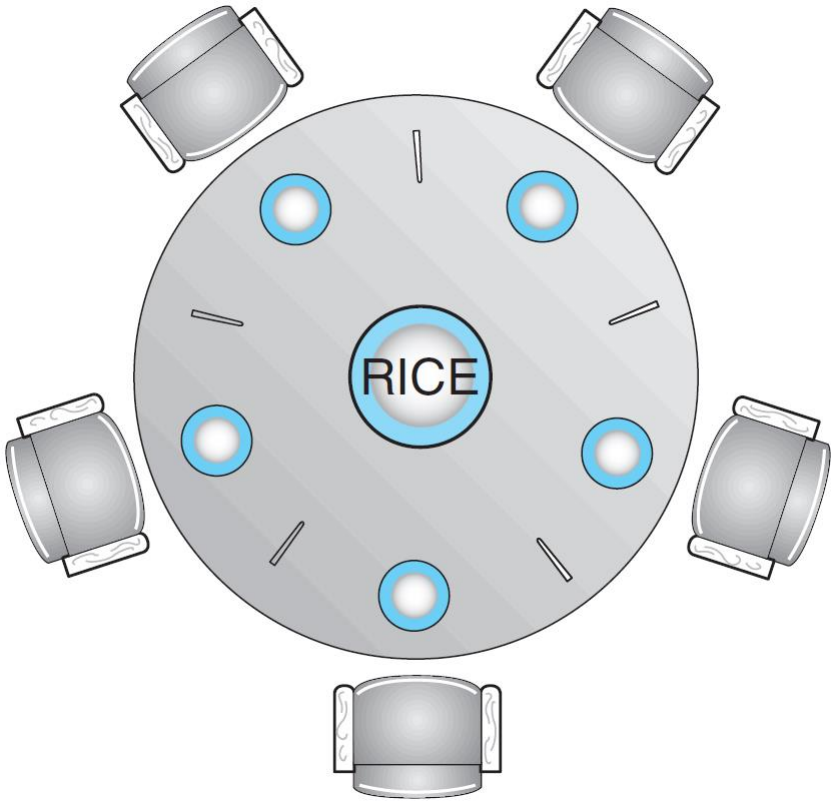
2. Tillåt att systemet hamnar i en deadlock, upptäcka det, och lösa det därefter.

3. Ignorera problemet helt och låtsas som om deadlocks inte kan uppstå i systemet.

Dining philosophers problem

Ett klassiskt synkroniseringsproblem.

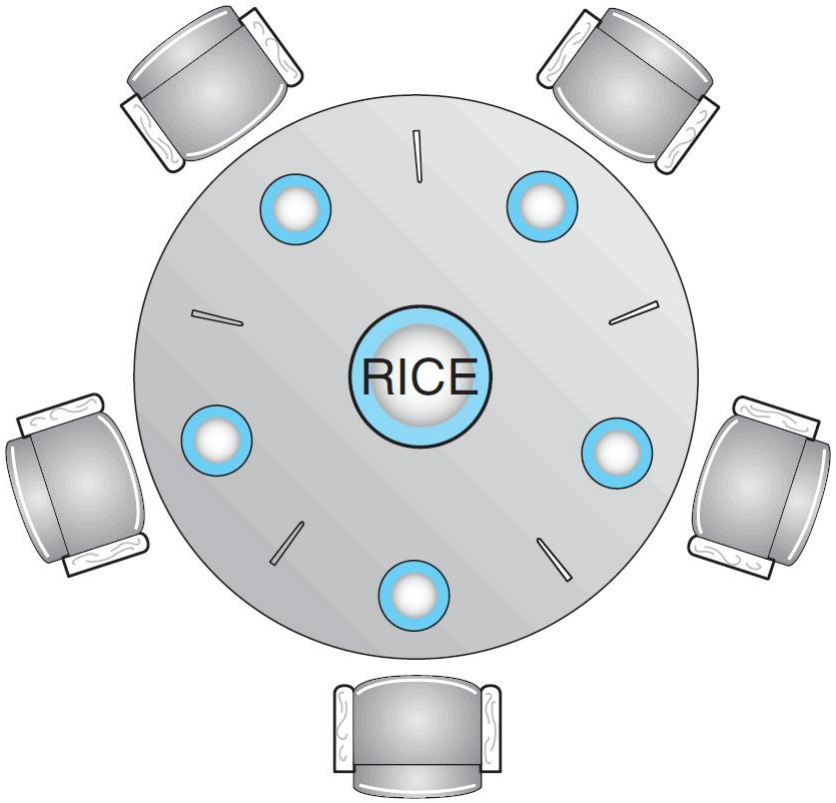
- Fem filosofer växlar mellan att äta ris och att filosofera.
- Det finns fem pinnar.
- För att äta behövs två närliggande pinnar.



A. Silberschatz, et al. Operating System Concepts. International Student Version. 9th ed. Hoboken, N.J.: John Wiley & Sons Inc, 2013. s. 272

Dining philosophers problem
Lösningar: ett första test

```
while (True):
    think()
    takeLeftFork()
    takeRightFork()
    eat()
    putLeftFork()
    putRightFork()
```

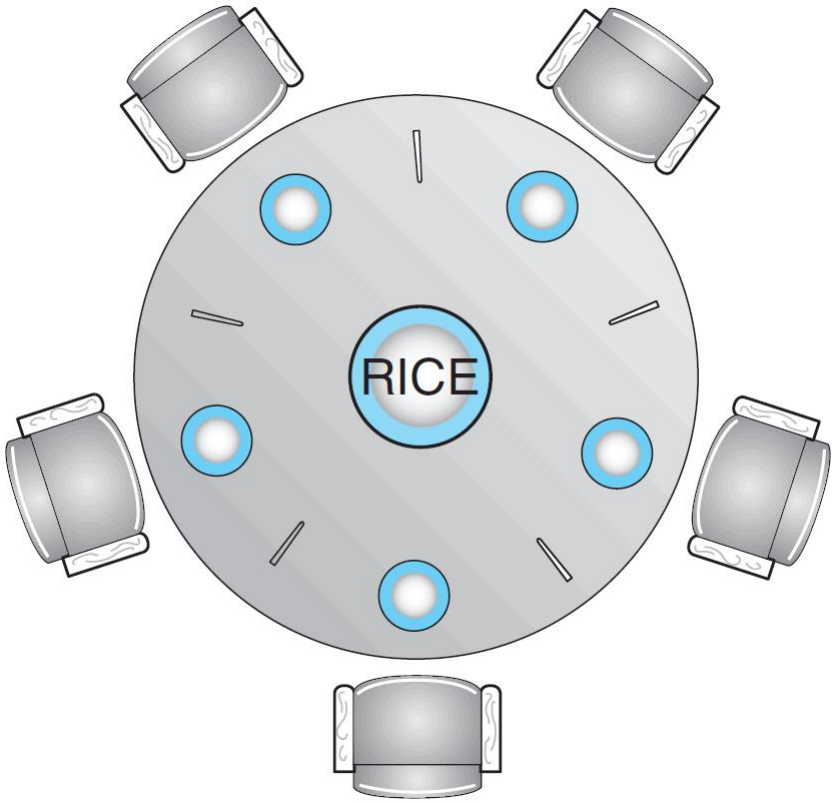


Problem: Ingen synkronisering. Funkar därför inte.

Deadlock om t.ex. alla tar sin vänstra pinne samtidigt.

Dining philosophers problem
Lösningar: strict alternation

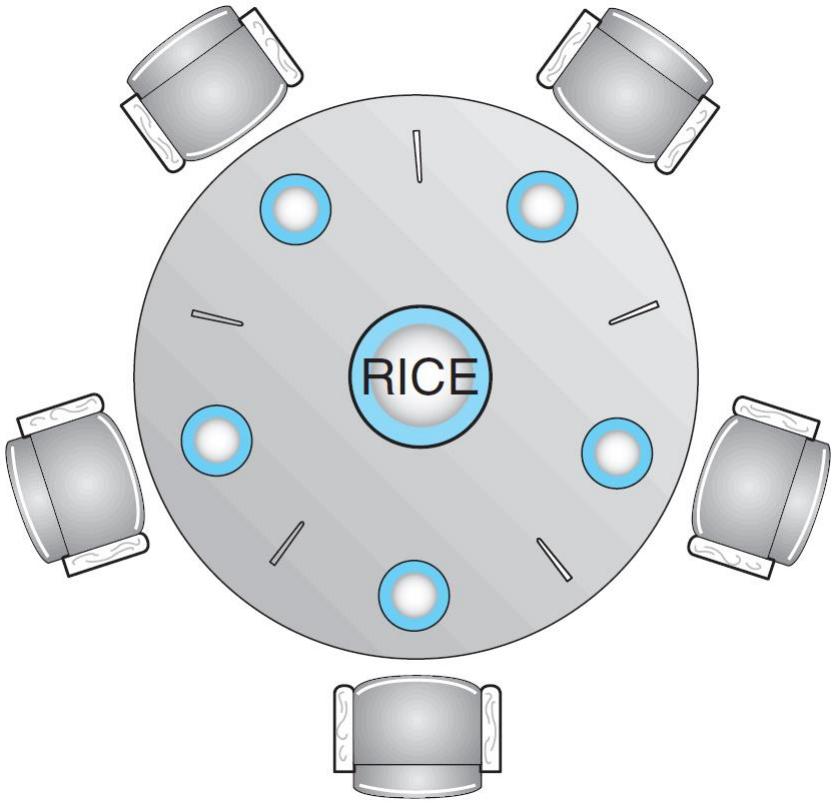
```
while (True):
   think()
   while (turn != me):
        pass
   takeLeftFork()
   takeRightFork()
   eat()
   putLeftFork()
   putRightFork()
   turn = next
```



Problem: Busy-waiting. En åt gången, kanske två med lite modifikation.

Dining philosophers problem
Lösningar: stäng av interrupts

```
while (True):
    think()
    disableInterrupts()
    takeLeftFork()
    takeRightFork()
    eat()
    putLeftFork()
    putRightFork()
    enableInterrupts()
```

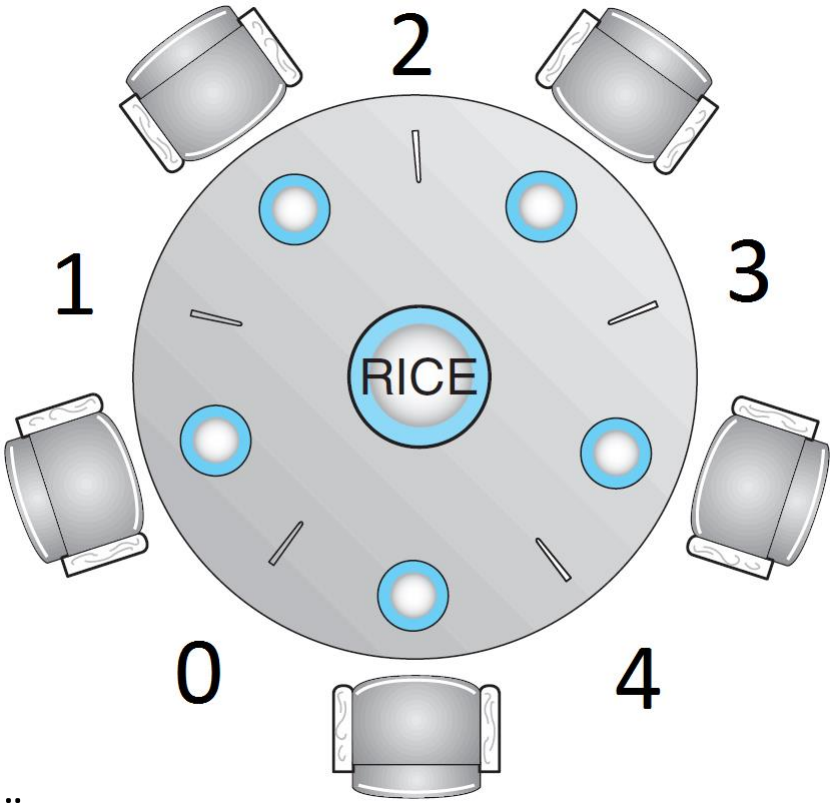


Problem: En åt gången.

Dining philosophers problem
Lösningar: resource hierarchy

```
while (True):
   think()
   while (me.forks < 2):
        takeForkWithLowestId()

   eat()
   putLeftFork()
   putRightFork()
```



Stabil lösning.

Problem: Programutvecklarna måste ta hänsyn till resurshierarkin. Resurser används ineffektivt.

Exempel i Python

- Två trådar och två globala variabler med lås för respektive.
  - Ena tråden ökar på variablerna med ett tal.
  - Den andra tråden ökar på variablerna med ett annat tal.

Synkronisering: Deadlocks Sammanfattning

- Deadlocks kan uppstå när flera processer konkurrerar om gemensamma resurser.
- Fyra krav: mutual exclusion, hold and wait, no preemption, circular wait.
- Förhindra deadlocks genom att säkerställa att åtminstone ett krav inte uppfylls.